



VIT[®]
B H O P A L
www.vitbhopal.ac.in

CSE 2006 - Programming in Java

Course Type: LP

Credits: 3

Prepared by

Dr Komarasamy G

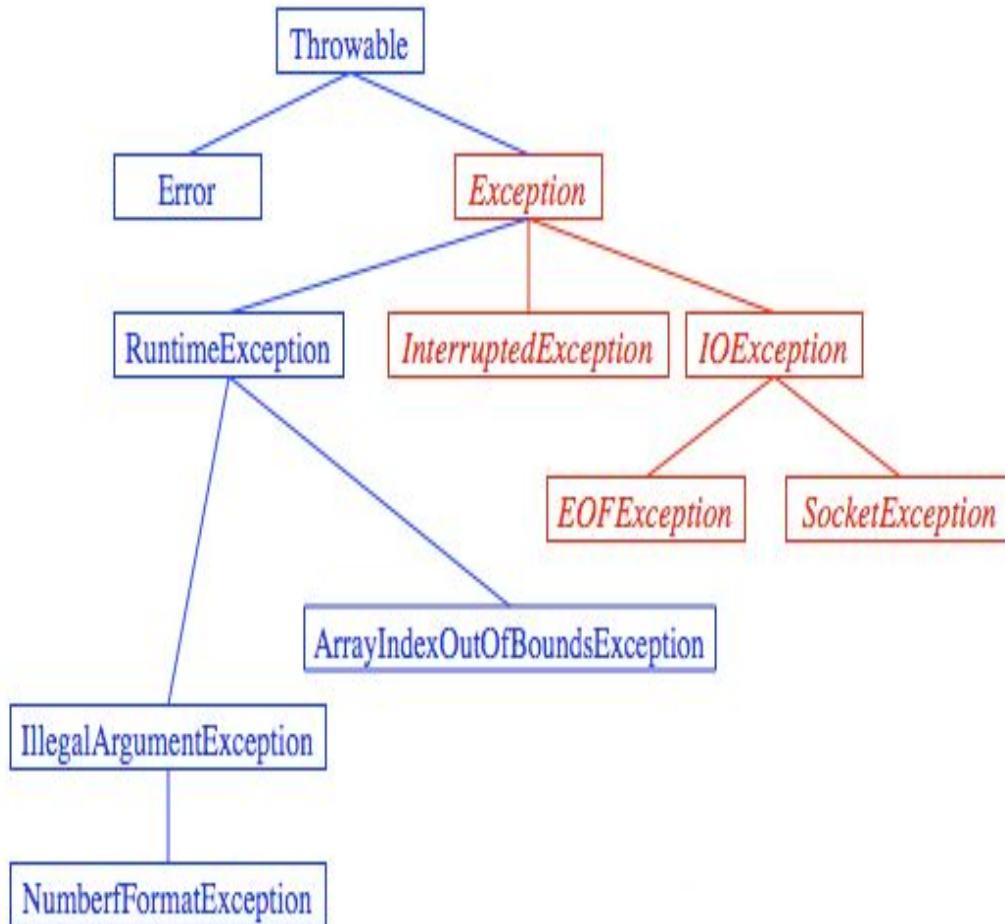
Senior Associate Professor

School of Computing Science and Engineering

VIT Bhopal University

- **Java Exception Handling**
- Java Exceptions, Java Exception Handling, Java try...catch, Java throw and throws, Java catch Multiple Exceptions, Java Annotations Types
- **Multithreading**
- Introduction, Thread Creations, Thread Life Cycle, Life Cycle Methods, Java Synchronization methods, User-defined packages.

What do you understand



What You Will Learn

Use
try,
throw,
catch



to

watch for
indicate
handle

} exceptions

How to process exceptions and failures.



Java Exception Handling

- The **Exception Handling** in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained.
- In this tutorial, we will learn about Java exceptions, it's types, and the difference between checked and unchecked exceptions.
- **What is Exception in Java?**
 - Dictionary Meaning: Exception is an abnormal condition.
 - In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.
- **What is Exception Handling?**
 - Exception Handling is a mechanism to handle runtime errors such as **ClassNotFoundException**, **IOException**, **SQLException**, **RemoteException**, etc.

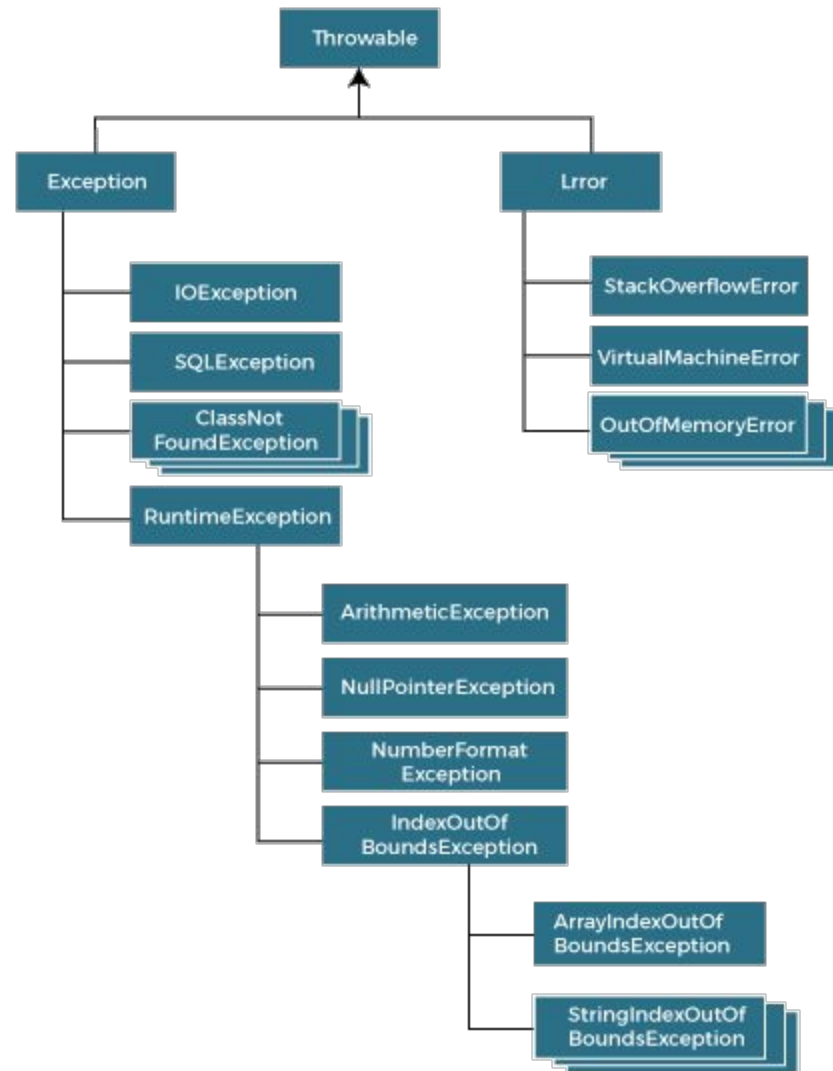
Java Exception Handling

- **Advantage of Exception Handling**

- The core advantage of exception handling is to maintain the normal flow of the application. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:
 - statement 1;
 - statement 2;
 - **Statement 3; //exception occurs**
 - statement 4;
 - statement 5;
- Suppose there are 5 statements in a Java program and an exception occurs at **statement 3**; the rest of the code will not be executed, i.e., statements 4 to 5 will not be executed.
- However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in Java.

Java Exception Handling

- **Hierarchy of Java Exception classes**
- The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Java Exception Handling

- **Types of Java Exceptions**
- There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:
 - **Checked Exception**
 - **Unchecked Exception**
 - **Error**

Java Exception Handling

- **Difference between Checked and Unchecked Exceptions**
- **1) Checked Exception**
- The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions.
- For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.
- **2) Unchecked Exception**
- The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.
- **3) Error**
- Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Handling

- **Java Exception Keywords**

- Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java Exception Handling

- ❑ An **exception** is an abnormal condition that arises in a code sequence at run time.
- ❑ In other words, an exception is a **run-time error**.

Exception-Handling Fundamentals

- ❑ Java exception is an object that describes an exceptional (that is, error) condition that has occurred in a piece of code.
- ❑ When an exceptional condition arises, an object representing that exception is created and *thrown* in the method that caused the error.
- ❑ Java exception handling is managed via five keywords:
try, catch, throw, throws and **finally**.
- ❑ Program statements that you want to monitor for exceptions are contained within a **try** block.

Java Exception Handling

This is the general form of an exception-handling block

```
try {  
    // Code that may throw an exception  
} catch (ExceptionType1 e1) {  
    // Handle exception of type ExceptionType1  
} catch (ExceptionType2 e2) {  
    // Handle exception of type ExceptionType2  
    // You can have multiple catch blocks for different  
exception types  
} finally {  
    // Code that will always execute (e.g., cleanup code)  
    // This block is optional  
}
```

Java Exception Handling

❑ Exception Types

- ❑ **Throwable** is at the top of the exception class hierarchy.
- ❑ Immediately below **Throwable** are two subclasses that partition exceptions into two distinct branches.
- ❑ One branch is headed by **Exception**.
- ❑ This class is used for exceptional conditions that user programs should catch.
- ❑ This is also the class that you will subclass to create your own custom exception types.
- ❑ There is an important subclass of **Exception**, called **Runtime Exception**.

Java Exception Handling

❑ Uncaught Exceptions

- The expression that intentionally causes a **divide-by-zero error**.

```
class Exc0 {  
    public static void main(String args[]) {  
        int d = 0;  
        int a = 42 / d;  
    }  
}
```

- ❑ When the Java run-time system detects the attempt to divide by zero
- ❑ it constructs a new exception object and then throws this exception.

Java Exception Handling

- Java Exception Handling Example
- Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```
public class JavaExceptionExample{  
    public static void main(String args[]){  
        try{  
            //code that may raise exception  
            int data=100/0;  
        }catch(ArithmeticException e){System.out.println(e);}  
        //rest code of the program  
        System.out.println("rest of the code...");  
    }  
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero  
rest of the code...
```

Common Scenarios of Java Exceptions

- There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where `ArithmeticException` occurs

- If we divide any number by zero, there occurs an `ArithmeticException`.
- `int a=50/0;//ArithmeticException`

2) A scenario where `NullPointerException` occurs

- If we have a null value in any variable, performing any operation on the variable throws a `NullPointerException`.
- `String s=null;`
- `System.out.println(s.length());//NullPointerException`

Common Scenarios of Java Exceptions

3) A scenario where `NumberFormatException` occurs

- If the formatting of any variable or number is mismatched, it may result into `NumberFormatException`. Suppose we have a string variable that has characters; converting this variable into digit will cause `NumberFormatException`.
- `String s="abc";`
- `int i=Integer.parseInt(s);//NumberFormatException`

4) A scenario where `ArrayIndexOutOfBoundsException` occurs

- When an array exceeds to its size, the `ArrayIndexOutOfBoundsException` occurs. there may be other reasons to occur `ArrayIndexOutOfBoundsException`. Consider the following statements.
- `int a[]=new int[5];`
- `a[10]=50; //ArrayIndexOutOfBoundsException`

try and catch example

```
class Exc2 {  
    public static void main(String args[]) {  
        int d, a;  
        try { // monitor a block of code.  
            d = 0;  
            a = 42 / d;  
            System.out.println("This will not be printed.");  
        } catch (ArithmeticException e)  
        { // catch divide-by-zero error  
            System.out.println("Division by zero.");  
        }  
        System.out.println("After catch statement.");  
    }  
}
```

try and catch

```
// File Name : ExceptTest.java
import java.io.*;
public class ExceptTest{

    public static void main(String args[]){
        try{
            int a[] = new int[2];
            System.out.println("Access element three :" + a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception thrown  :" + e);
        }
        System.out.println("Out of the block");
    }
}
```

Output:

Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 3
Out of the block

Multiple catch Clauses

```
class MultiCatch {  
    public static void main(String args[]) {  
        try {  
            int a = args.length;  
            System.out.println("a = " + a);  
            int b = 42 / a;  
            int c[] = { 1 };  
            c[42] = 99;  
        } catch(ArithmeticException e) {  
            System.out.println("Divide by 0: " + e);  
        } catch(ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index oob: " + e);  
        }  
        System.out.println("After try/catch blocks.");  
    }  
}
```

Multiple catch Clauses

Here is the output generated by running it both ways:

```
C:\>java MultiCatch
```

```
a = 0
```

```
Divide by 0: java.lang.ArithmeticException: / by zero
```

```
After try/catch blocks.
```

```
C:\>java MultiCatch TestArg
```

```
a = 1
```

```
Array index oob:
```

```
java.lang.ArrayIndexOutOfBoundsException
```

```
After try/catch blocks.
```

throw

```
class ThrowDemo {  
    static void demoproc() {  
        try {  
            throw new NullPointerException("demo");  
        } catch (NullPointerException e)  
        {  
            System.out.println("Caught inside demoproc.");  
            throw e; // rethrow the exception  
        }  
    }  
    public static void main(String args[]) {  
        try {    demoproc();  
        } catch (NullPointerException e)  
        { System.out.println("Recaught: " + e); }  
    }  
}
```

output:

Caught inside demoproc.

Recaught: java.lang.NullPointerException: demo

throw

- This program gets two chances to deal with the same error.
- First, **main()** sets up an exception context and then calls **demoproc()**.
- The **demoproc()** method then sets up another exception-handling context and immediately throws a new instance of **NullPointerException**, which is caught on the next line.
- The exception is then re-thrown.

throws

- A throws clause **lists the types** of exceptions that a method might throw.
- All other exceptions that a method can throw must be declared in the **throws clause**.
- **If** they are not, a compile-time error will result.

Type method-name(parameter-list) throws exception-list
{ // body of method }

- Here, ***exception-list*** is a comma-separated list of the exceptions that a method can throw.

throws

// This program contains an error and will not compile.

```
class ThrowsDemo {  
    static void throwOne() {  
        System.out.println("Inside throwOne.");  
        throw new IllegalAccessException("demo");  
    }  
    public static void main(String args[])  
    {    throwOne();    }  
}
```

throws

// This is now correct

```
class ThrowsDemo {  
    static void throwOne() throws IllegalAccessException  
    {  
        System.out.println("Inside throwOne.");  
        throw new IllegalAccessException("demo");  
    }  
    public static void main(String args[]) {  
        try {  
            throwOne();  
        }  
        catch (IllegalAccessException e) {  
            System.out.println("Caught " + e);  
        }  
    }  
}
```

Output:

inside throwOne

caught java.lang.IllegalAccessException: demo

Finally

- finally creates a block of code that will be executed after a try/catch block has completed and before the code following the try/catch block.
- The finally block will **execute whether or not an exception is thrown**.

```
finally
{
    System.out.println("procA's finally");
}
```

Java's Built-in Exceptions

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type
ClassCastException	Invalid cast.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, eg: waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state
IllegalThreadStateException	Requested operation not compatible with current thread state
IndexOutOfBoundsException	Some type of index is out-of-bounds
NegativeArraySizeException	Array created with a negative size
NullPointerException	Invalid use of a null reference
UnsupportedOperationException	An unsupported operation was encountered

Creating Your Own Exception Subclasses

- To create your own exception types to handle situations specific to your applications.
- Define a subclass of Exception, subclasses don't need to actually implement anything.
- The Exception class does not define any methods of its own.
- It inherits those methods provided by Throwable

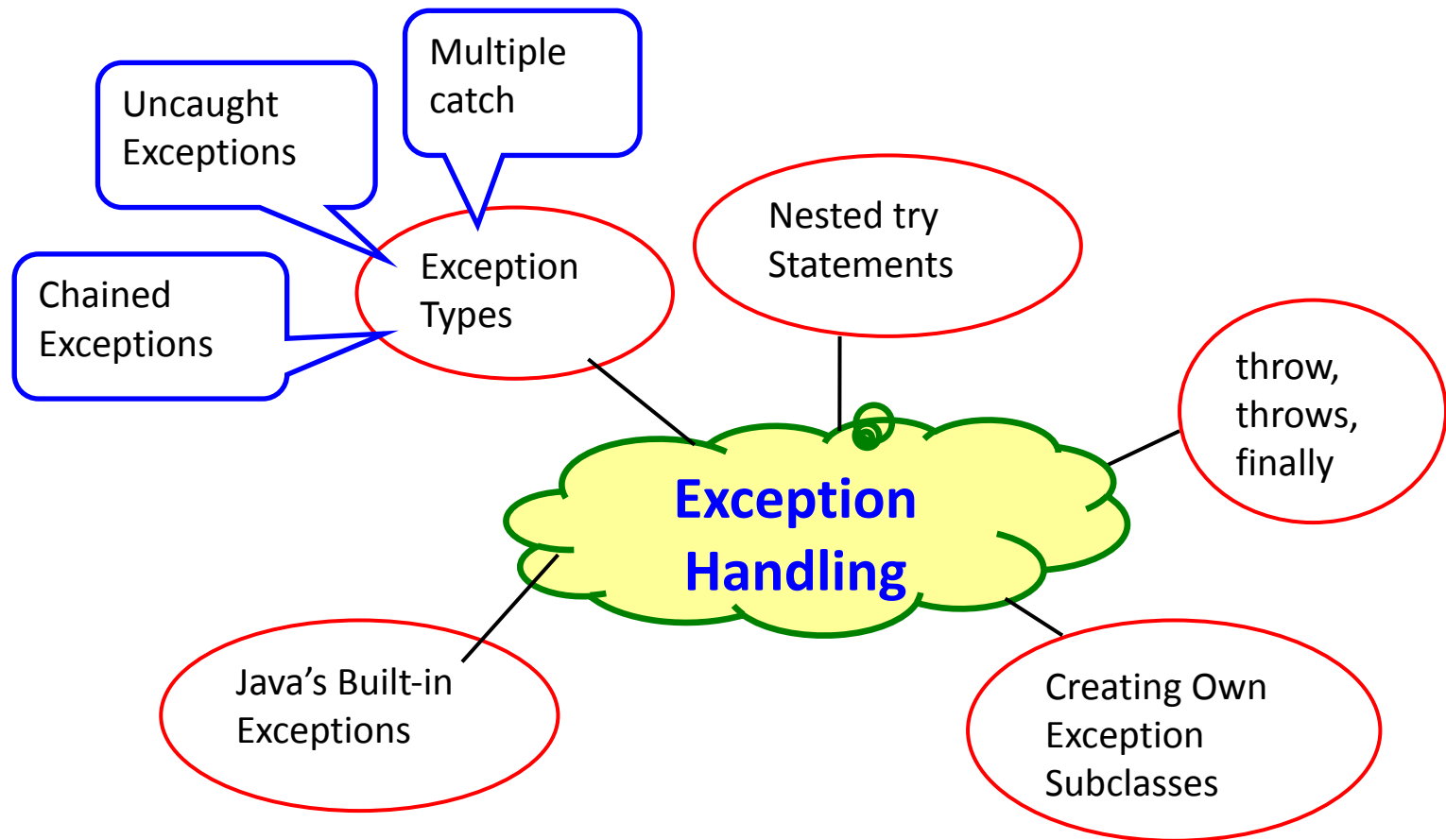
- Throwable fillInStackTrace()
- Throwable getCause()
- String getLocalizedMessage()
- String getMessage()
- StackTraceElement[]
getStackTrace()

- Throwable initCause(Throwable *causeExc*)
- void printStackTrace()
- void printStackTrace(PrintStream *stream*)
- void printStackTrace(PrintWriter *stream*)
- void setStackTrace(StackTraceElement
elements[])
- String toString()

Chained Exceptions

- The chained exception feature allows you to associate another exception with an exception.
- This second exception describes the cause of the first exception.
- Two constructors and two methods to Throwable.
 - `Throwable(Throwable causeExc)`
 - `Throwable(String msg, Throwable causeExc)`
 - `Throwable getCause( )`
 - `Throwable initCause(Throwable causeExc)`

Mind map



MCQ Questions

Which of the following is the correct keyword to handle an exception in Java?

- a) catch
- b) try
- c) throw
- d) finally

b) try

Which of these is a checked exception?

- a) NullPointerException
- b) ArithmeticException
- c) IOException
- d) ArrayIndexOutOfBoundsException

c) IOException

What happens if an exception is not caught?

- a) The program will stop abruptly and crash.
- b) The program will continue without any issue.
- c) The exception is ignored.
- d) The program will print "Exception ignored."

a) The program will stop abruptly and crash.

MCQ Questions

What is the output of the following code?

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println("Caught ArithmeticException");  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

d) "Caught ArithmeticException" followed by "Finally block executed"

- a) No output
- b) Only "Caught ArithmeticException"
- c) Only "Finally block executed"
- d) "Caught ArithmeticException" followed by "Finally block executed"

MCQ Questions

Which of these statements about the finally block is true?

- a) The finally block is executed only when an exception is thrown.
- b) The finally block is executed only if no exception is thrown.
- c) The finally block is executed only after the catch block.
- d) The finally block is executed whether an exception is thrown or not.

d) The finally block is executed whether an exception is thrown or not.

What will be the result of executing the following code?

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            return;  
        } finally {  
            System.out.println("Finally executed");  
        }  
    }  
}
```

c) "Finally executed" will be printed.

- a) No output
- b) The program will return without executing the finally block.
- c) "Finally executed" will be printed.
- d) A compile-time error will occur.

More details

- <https://www.javatpoint.com/exception-handling-in-java>

MCQ questions

- <https://www.sanfoundry.com/java-mcqs-exceptional-handling-basics/>
- <https://www.examveda.com/java-program/practice-mcq-question-on-excptions/>
- <https://www.geeksforgeeks.org/questions/java-exception-handling-questions-2/>
- <https://www.javamadesoeasy.com/2015/10/exception-handling-quiz-in-java.html>